

# Conclusion to SQLAlchemy

## Learning Goals

- Use an external library to simplify tasks from earlier ORM lessons.
  - Manage database tables and schemas without ever writing SQL through Alembic.
  - Use SQLAlchemy to create, read, update and delete records in a SQL database.
- 

## Key Vocab

- **Schema:** the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
  - **Persist:** save a schema in a database.
  - **Engine:** a Python object that translates SQL to Python and vice-versa.
  - **Session:** a Python object that uses an engine to allow us to programmatically interact with a database.
  - **Transaction:** a strategy for executing database statements such that the group succeeds or fails as a unit.
  - **Migration:** the process of moving data from one or more databases to one or more target databases.
- 

## Conclusion

We have just explored the world of ORMs: Object-Relational Mappers. When creating applications with Python (and later, [Flask](https://flask.palletsprojects.com/en/2.2.x/) ) (<https://flask.palletsprojects.com/en/2.2.x/>), we use an ORM called SQLAlchemy, integrated into our projects as a Python package. SQLAlchemy creates a link between Python and our database, which means it can take care of translating statements we write using Python into commands the database understands. It makes our work easier and faster.

Over the past several lessons, we covered the basic mechanics of SQLAlchemy, including connecting to a database, creating tables and finding/saving data. Of course, we then practiced employing SQLAlchemy methods on short-term databases in memory and more durable SQLite databases. We also learned that instead of making individual, manual changes to databases, we can use Alembic migrations, which apply database changes as an organized structure. Migrations serve as a sort of version control system (like Git!) for our database, so we covered how they work and how to write our own. Ultimately, we saw how we can execute CRUD actions — create, read, update, delete — for our applications with SQLAlchemy, an essential skill we'll be using as we create web applications in Phase 4.

We have Python knowledge, we have database knowledge and now we know how to use the ORM layer to communicate between the two. In the next Canvas module, we will continue to explore SQLAlchemy as we build relationships between tables.

---

## Resources

- [Python 3.8.13 Documentation - Python](https://docs.python.org/3.8/) ↗
- [sqlite3 — DB-API 2.0 interface for SQLite databases - Python](https://docs.python.org/3/library/sqlite3.html) ↗
- [SQLAlchemy ORM Documentation - SQLAlchemy](https://docs.sqlalchemy.org/en/14/orm/) ↗
- [SQLAlchemy — Python Tutorial - Towards Data Science](https://towardsdatascience.com/sqlalchemy-python-tutorial-79a577141a91) ↗
- [Alembic 1.8.1 Documentation - SQLAlchemy](https://alembic.sqlalchemy.org/en/latest/) ↗
- [Tutorial \(Alembic\) - SQLAlchemy](https://alembic.sqlalchemy.org/en/latest/tutorial.html) ↗
- [Flask 2.2.x Documentation](https://flask.palletsprojects.com/en/2.2.x/) ↗